Listing 9.3 Configuring Clerk middleware

The main middleware function allows you to protect routes in your application based on authentication status.

```
import { clerkMiddleware, createRouteMatcher } from "@clerk/nextjs/server";

const isProtectedRoute =
createRouteMatcher(["/chat(.*)"]);

export default clerkMiddleware(async (auth, req) => {
  if (isProtectedRoute(req)) await auth.protect();

  const corsResponse = handleCORS(req);
  if (corsResponse instanceof NextResponse) {
    return corsResponse;
  }
  const rateLimitResponse = rateLimit(req);
  if (rateLimitResponse instanceof NextResponse) {
    return rateLimitResponse;
  }
  return NextResponse.next();
});

export const config = {
  matcher: [
    // Skip Next.js internals and all static files, unless found in search params
    '/((?!_next|[^?]*\\.(?:html?|css|js(?!on)
|jpe?g|webp|png|gif|svg|ttf|woff2?|
ico|csv|docx?|xlsx?|zip|webmanifest)).*)',
    // Always run for API routes
    '/(api|trpc)(.*)',
  ]
};
```

We are using a route matcher; if it matches, the auth.protect() method will be invoked.

With this middleware in place, Clerk.js will protect all routes under the `/chat` endpoint (including the `chat/api` routes) from unauthenticated requests. The last step is to make a redirect to the sign-in page whenever a user lands on our site.

CREATING THE SIGN-IN PAGE AND UNAUTHENTICATED REDIRECTS

To manage the initial user experience, we'll implement a redirect on the index page. The goal is to check if the user is authenticated. If not, we'll redirect them to the `/sign-in` page. If they are authenticated, we'll redirect them to the `/chat` page.

Listing 9.4 Redirecting to authentication page if not signed in

```
'use client';
import { useUser } from "@clerk/nextjs";
import { useRouter } from 'next/navigation'; // Changed import
import { useEffect } from 'react';

export default function IndexPage() {
```